DRL Assignment -PART1-MAB:

Adaptive Treatment Recommendation System using Multi Armed Bandit Learning.

Team # 116

Student Name	Student ID
PRASAD SHIVAJI KULKARNI	2025aa05444@wilp.bits-pilani.ac.in
SHELAR SACHIN KRISHNA	2025aa05387@wilp.bits-pilani.ac.in
POWAR SAGAR GANPATI	2025aa05421@wilp.bits-pilani.ac.in
SUJEET KUMAR YADAV	2025aa05326@wilp.bits-pilani.ac.in
JAMDAR SARTHAK VITHALDAS	2025aa05112@wilp.bits-pilani.ac.in

Student Collab Image:

L–R: Sagar, Sarthak, Sujeet
Prasad, Sachin

Student Contributions Sheet:

Student Name	Student Contribution
PRASAD SHIVAJI KULKARNI	Generated synthetic patient-treatment dataset and implemented medicine probability logic. Ensured reproducibility and validated generated data.
SHELAR SACHIN KRISHNA	Implemented Immediate Exploitation strategy and reward computation. Performed simulation runs and analyzed cumulative rewards.
POWAR SAGAR GANPATI	Developed Epsilon-Greedy strategy with multiple exploration rates. Compared exploration-exploitation tradeoffs through experiments.
SUJEET KUMAR YADAV	Executed solution in virtual machine and captured required screenshots. Prepared final documentation, result validation, and report compilation.
JAMDAR SARTHAK VITHALDAS	Implemented UCB1 algorithm and cumulative reward tracking. Created graphs, visualizations, and comparative performance analysis.

python code for: Part #1 - MAB

```
"""
================================================================================
BITS WILP - Deep Reinforcement Learning | Lab Assignment 1 - Part 1 (MAB)
Adaptive Treatment Recommendation System using Multi-Armed Bandit Learning
================================================================================
Group Number : 116
Submission File: 116_MAB.py

Tasks:
  1. Synthetic patient-treatment dataset generation
  2. Immediate Exploitation Strategy
  3. Controlled Clinical Trial (Epsilon-Greedy: 10%, 1%, 50%)
  4. Confidence-Based Strategy (UCB1)
  5. Comparative Analysis (plot + summary)
================================================================================
"""

import os
import platform
import random
import socket
from datetime import datetime

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# ================================================================================
# Virtual Lab Metadata (printed at execution start per assignment requirement)
# ================================================================================

EXECUTION_TIMESTAMP = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
HOSTNAME = socket.gethostname()
VM_ID = (
    os.environ.get("VM_ID")
    or os.environ.get("COMPUTERNAME")
    or platform.node()
    or HOSTNAME
)

print("=" * 70)
print("VIRTUAL LAB EXECUTION METADATA")
print("=" * 70)
print(f"Execution Timestamp : {EXECUTION_TIMESTAMP}")
print(f"Virtual Machine ID  : {VM_ID}")
print(f"Hostname            : {HOSTNAME}")
print(f"Platform            : {platform.platform()}")
print("=" * 70)

# ================================================================================
# Assignment Parameters — Group G = 116
# ================================================================================

GROUP_NO = 116

random.seed(GROUP_NO)
np.random.seed(GROUP_NO)
```

```
# K = (G mod 3) + 5
K = (GROUP_NO % 3) + 5
TOTAL_PATIENTS = 1000

# Pi = 0.4 + ((G + i) mod 6) * 0.07
HIDDEN_PROBABILITIES = [
    round(0.4 + (((GROUP_NO + i) % 6) * 0.07), 2) for i in range(K)
]

# True best medicine index (highest hidden success probability)
TRUE_BEST_MEDICINE = int(np.argmax(HIDDEN_PROBABILITIES))


# ============================================================================
# Task 1: Dataset Design
# ============================================================================

def build_base_dataset():
    """
    Create the static portion of the synthetic dataset (1000 patients).

    Columns:
        patient_id      : 0 to 999
        severity_score  : (patient_id mod 5) + 1  -> values 1 to 5

    During each bandit algorithm run, assigned_medicine, clinical_outcome,
    and utility_score are filled dynamically.
    """
    df = pd.DataFrame()
    df["patient_id"] = np.arange(TOTAL_PATIENTS)
    df["severity_score"] = (df["patient_id"] % 5) + 1
    return df


def simulate_patient(medicine_id, severity):
    """
    Simulate one patient treatment episode.

    clinical_outcome:
        Bernoulli draw with probability P_i (hidden success rate of medicine i).

    utility_score:
        clinical_outcome * (1 - severity / 10)
        Recovery on mild patients yields higher utility than on critical patients.

    Returns
    -------
    clinical_outcome : int (0 or 1)
    utility_score    : float
    """
    success_probability = HIDDEN_PROBABILITIES[medicine_id]
    clinical_outcome = int(np.random.binomial(n=1, p=success_probability))
    utility_score = clinical_outcome * (1.0 - severity / 10.0)
    return clinical_outcome, utility_score


def reset_random_state():
    """Reset RNG so every strategy faces the same stochastic environment."""
    random.seed(GROUP_NO)
    np.random.seed(GROUP_NO)
```

```
def run_bandit_simulation(select_medicine_fn, strategy_name):
    """
    Generic simulation loop for 1000 patients.

    Parameters
    ----------
    select_medicine_fn : callable
        Function(patient_id, state) -> medicine index.
        `state` is a mutable dict holding bandit statistics and logs.

    strategy_name : str
        Label used when storing results.

    Returns
    -------
    results_df : DataFrame
        Full dataset with dynamic columns populated.
    cumulative_rewards : list
        Cumulative utility_score after each patient.
    convergence_step : int or None
        First patient index where estimated best arm equals true best arm.
    """
    reset_random_state()

    df = build_base_dataset()
    df["assigned_medicine"] = -1
    df["clinical_outcome"] = 0
    df["utility_score"] = 0.0

    state = {
        "successes": np.zeros(K, dtype=float),  # sum of clinical_outcome per arm
        "counts": np.zeros(K, dtype=float),     # pulls per arm
        "cumulative_rewards": [],
        "convergence_step": None,
    }

    cumulative = 0.0

    for patient_id in range(TOTAL_PATIENTS):
        severity = int(df.loc[patient_id, "severity_score"])
        medicine = select_medicine_fn(patient_id, state)

        clinical_outcome, utility_score = simulate_patient(medicine, severity)

        # Update bandit statistics using clinical_outcome (per assignment)
        state["successes"][medicine] += clinical_outcome
        state["counts"][medicine] += 1

        cumulative += utility_score
        state["cumulative_rewards"].append(cumulative)

        df.loc[patient_id, "assigned_medicine"] = medicine
        df.loc[patient_id, "clinical_outcome"] = clinical_outcome
        df.loc[patient_id, "utility_score"] = utility_score

        # Track earliest step where estimated best matches true best (all arms tried)
        if state["convergence_step"] is None and np.all(state["counts"] > 0):
            estimated_rates = state["successes"] / state["counts"]
            if int(np.argmax(estimated_rates)) == TRUE_BEST_MEDICINE:
                state["convergence_step"] = patient_id + 1
```

```
    df.attrs["strategy_name"] = strategy_name
    return df, state["cumulative_rewards"], state["convergence_step"]


# ============================================================================
# Task 2: Immediate Exploitation Strategy
# ============================================================================

def immediate_exploitation_select(patient_id, state):
    """
    Phase 1 (patients 0 .. 10*K-1): test each medicine exactly 10 times.
    Phase 2: always prescribe the medicine with highest empirical success rate
             (clinical_outcome rate).
    """
    phase1_patients = 10 * K

    if patient_id < phase1_patients:
        # Round-robin: medicine 0 gets patients 0-9, medicine 1 gets 10-19, ...
        return patient_id // 10

    if "locked_medicine" not in state:
        rates = np.divide(
            state["successes"],
            state["counts"],
            out=np.zeros(K),
            where=state["counts"] > 0,
        )
        state["locked_medicine"] = int(np.argmax(rates))
        state["empirical_rates_after_explore"] = rates.copy()

    return state["locked_medicine"]


# ============================================================================
# Task 3: Controlled Clinical Trial — Epsilon-Greedy
# ============================================================================

def make_epsilon_greedy_select(epsilon):
    """
    Factory for epsilon-greedy action selection.

    With probability epsilon: explore (random medicine).
    With probability 1-epsilon: exploit (highest empirical clinical success rate).

    First K patients each try a distinct medicine once to avoid division by zero.
    """

    def select(patient_id, state):
        if patient_id < K:
            return patient_id

        if random.random() < epsilon:
            return random.randint(0, K - 1)

        rates = np.divide(
            state["successes"],
            state["counts"],
            out=np.zeros(K),
            where=state["counts"] > 0,
        )
```

```
        return int(np.argmax(rates))

    return select


# ============================================================================
# Task 4: UCB1 — Confidence-Based Strategy
# ============================================================================

def ucb1_select(patient_id, state):
    """
    UCB1 index for Bernoulli bandits (clinical_outcome as reward signal):

        UCB_i = (successes_i / counts_i) + sqrt(2 * ln(t) / counts_i)

    where t is total number of pulls so far.
    """
    if patient_id < K:
        return patient_id

    t = patient_id  # total pulls completed before this decision
    ucb_values = []

    for arm in range(K):
        avg_reward = state["successes"][arm] / state["counts"][arm]
        confidence = np.sqrt((2.0 * np.log(t)) / state["counts"][arm])
        ucb_values.append(avg_reward + confidence)

    return int(np.argmax(ucb_values))


# ============================================================================
# Task 5 helpers: stability metric (variance of step-wise reward increments)
# ============================================================================

def reward_increment_variance(cumulative_rewards):
    """
    Measure stability: lower variance of per-step utility increments
    implies smoother cumulative reward growth.
    """
    increments = np.diff([0.0] + cumulative_rewards)
    return float(np.var(increments))


# ============================================================================
# Main execution
# ============================================================================

def print_task1_summary(base_df):
    """Display Task 1 required outputs."""
    print("\n" + "=" * 70)
    print("TASK 1: DATASET DESIGN")
    print("=" * 70)
    print(f"Group Number (G)             : {GROUP_NO}")
    print(f"Total Medicines (K)          : {K}")
    print(f"True Best Medicine (hidden)   : {TRUE_BEST_MEDICINE}")
    print("\nHidden Success Probabilities:")
    for i, prob in enumerate(HIDDEN_PROBABILITIES):
        marker = "  <-- highest" if i == TRUE_BEST_MEDICINE else ""
        print(f"  Medicine {i}: P = {prob:.2f}{marker}")
```

```
    print("\nFirst 10 rows of base dataset (patient_id, severity_score):")
    print(base_df[["patient_id", "severity_score"]].head(10).to_string(index=False))


def main():
    base_df = build_base_dataset()
    print_task1_summary(base_df)

    # --- Run all strategies ---
    print("\n" + "=" * 70)
    print("RUNNING BANDIT STRATEGIES (1000 patients each)")
    print("=" * 70)

    strategies = [
        ("Immediate Exploitation", immediate_exploitation_select),
        ("Epsilon-Greedy 10%", make_epsilon_greedy_select(0.10)),
        ("Epsilon-Greedy 1%", make_epsilon_greedy_select(0.01)),
        ("Epsilon-Greedy 50%", make_epsilon_greedy_select(0.50)),
        ("UCB1", ucb1_select),
    ]

    results = {}

    for name, selector in strategies:
        print(f"\n>>> {name} ...")
        df, cum_rewards, conv_step = run_bandit_simulation(selector, name)
        final_reward = cum_rewards[-1]
        stability = reward_increment_variance(cum_rewards)

        results[name] = {
            "df": df,
            "cumulative_rewards": cum_rewards,
            "final_reward": final_reward,
            "convergence_step": conv_step,
            "stability_var": stability,
        }

        print(f"    Final cumulative utility_score : {final_reward:.4f}")
        print(f"    Convergence step (est. best)   : {conv_step}")
        print(f"    Reward increment variance      : {stability:.6f}")

        if name == "Immediate Exploitation":
            rates = df.loc[: 10 * K - 1].groupby("assigned_medicine")["clinical_outcome"].mean()
            locked = int(df.loc[10 * K :, "assigned_medicine"].mode().iloc[0])
            print("    Empirical success rates after 10 trials/arm:")
            for med in range(K):
                rate = rates.get(med, 0.0)
                print(f"      Medicine {med}: {rate:.4f}")
            print(f"    Locked medicine for remaining patients: {locked}")

    # Show sample rows after one strategy (epsilon 10% as representative)
    sample_df = results["Epsilon-Greedy 10%"]["df"]
    print("\nSample rows after Epsilon-Greedy 10% (first 10 with all columns):")
    print(
        sample_df[
            [
                "patient_id",
                "severity_score",
                "assigned_medicine",
                "clinical_outcome",
                "utility_score",
```

```
            ]
        ]
        .head(10)
        .to_string(index=False)
    )

    # --- Task 3: exploration rate analysis ---
    print("\n" + "=" * 70)
    print("TASK 3: EXPLORATION RATE ANALYSIS")
    print("=" * 70)
    for eps_label in ["Epsilon-Greedy 10%", "Epsilon-Greedy 1%", "Epsilon-Greedy 50%"]:
        r = results[eps_label]
        print(
            f"{eps_label:22s} | Final reward: {r['final_reward']:.4f} | "
            f"Convergence: {r['convergence_step']} | Stability var: {r['stability_var']:.6f}"
        )
    print(
        "\nObservation: 1% exploration exploits aggressively (less discovery of better arms); "
        "50% exploration wastes many pulls on suboptimal medicines; 10% balances discovery "
        "and exploitation for this clinical trial setting."
    )

    # --- Final reward table ---
    print("\n" + "=" * 70)
    print("FINAL CUMULATIVE REWARDS (utility_score)")
    print("=" * 70)
    for name in results:
        print(f"  {name:28s}: {results[name]['final_reward']:.4f}")

    best_reward_strategy = max(results, key=lambda n: results[n]["final_reward"])
    fastest_convergence = min(
        (n for n in results if results[n]["convergence_step"] is not None),
        key=lambda n: results[n]["convergence_step"],
    )
    most_stable = min(results, key=lambda n: results[n]["stability_var"])

    # --- Task 5: Plot the Graph.
# ================================================================================
    plt.figure(figsize=(12, 6))
    for name in results:
        plt.plot(results[name]["cumulative_rewards"], label=name)
    plt.xlabel("Number of Patients")
    plt.ylabel("Cumulative Reward (utility_score)")
    plt.title(f"MAB Strategy Comparison — Group {GROUP_NO} (K={K} medicines)")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plot_path = os.path.join(
        os.path.dirname(os.path.abspath(__file__)),
        f"group_{GROUP_NO}_cumulative_reward.png",
    )
    plt.savefig(plot_path, dpi=150)
    print(f"\nPlot saved to: {plot_path}")
    plt.show()

    # --- Task 5: Comparative answers ---
    print("\n" + "=" * 70)
    print("TASK 5: COMPARATIVE ANALYSIS — ANSWERS")
    print("=" * 70)
    print(
        f"1. Highest cumulative reward after 1000 patients: {best_reward_strategy} "
```

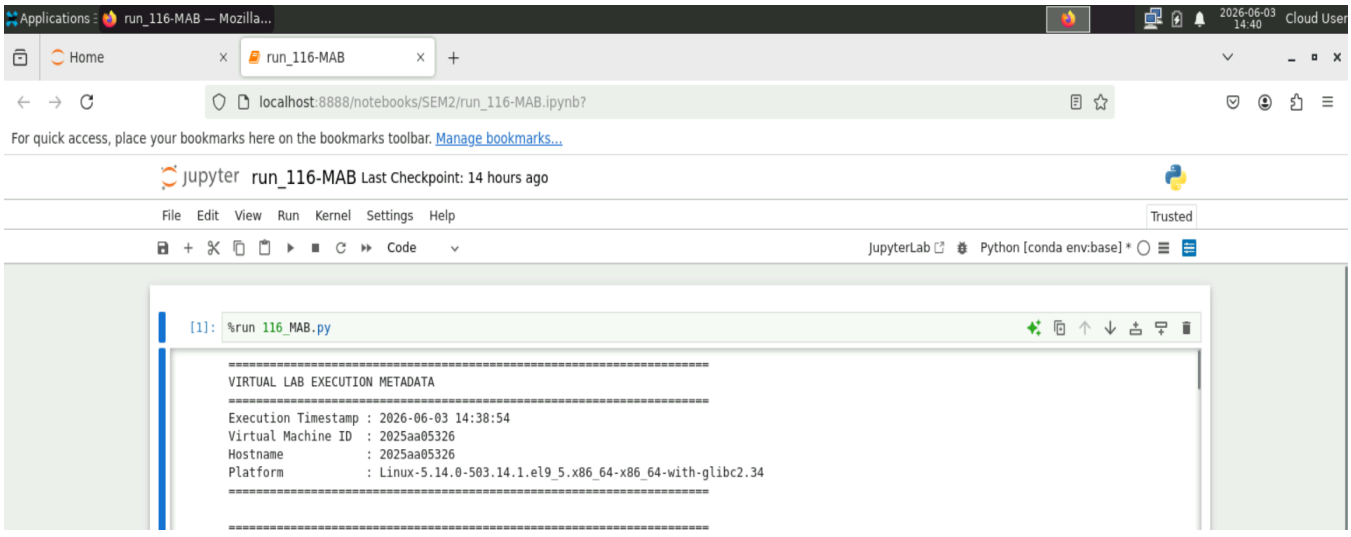
```
        f"({results[best_reward_strategy]['final_reward']:.4f})"
    )
    print(
        f"2. Fastest identification of best medicine: {fastest_convergence} "
        f"(step {results[fastest_convergence]['convergence_step']})"
    )
    print(
        f"3. Most stable performance (lowest reward-increment variance): {most_stable} "
        f"(variance = {results[most_stable]['stability_var']:.6f})"
    )
    print(
        "4. Safest real-world recommendation: UCB1 or Epsilon-Greedy 10%. "
        "UCB1 automatically favors under-sampled medicines then shifts toward evidence; "
        "10% epsilon-greedy retains controlled exploration without the heavy waste of 50% "
        "exploration. Immediate exploitation risks locking onto a suboptimal medicine after "
        "only 10 trials per arm."
    )

    print("\n" + "-" * 70)
    print("COMPARATIVE SUMMARY (3–5 sentences)")
    print("-" * 70)
    print(
        f"For group {GROUP_NO}, medicine {TRUE_BEST_MEDICINE} has the highest hidden "
        f"recovery probability ({HIDDEN_PROBABILITIES[TRUE_BEST_MEDICINE]:.2f}). "
        f"{best_reward_strategy} achieved the largest total utility_score, while "
        f"{fastest_convergence} identified the best arm earliest. "
        f"{most_stable} showed the smoothest cumulative reward curve. "
        "Epsilon-greedy at 1% behaves nearly as a pure exploit strategy and may miss "
        "better treatments; at 50% it over-explores and sacrifices patient utility. "
        "For hospital deployment, UCB1 or moderate epsilon-greedy (10%) is preferable "
        "because they limit persistent suboptimal prescribing while still gathering evidence."
    )


if __name__ == "__main__":
    main()
```

—----------Python code End—------

Screen Shot of Output from Virtual Lab: Time stamp:



All TASKs Output:

```
======================================================================
TASK 1: DATASET DESIGN
======================================================================
Group Number (G)            : 116
Total Medicines (K)         : 7
True Best Medicine (hidden)   : 3

Hidden Success Probabilities:
  Medicine 0: P = 0.54
  Medicine 1: P = 0.61
  Medicine 2: P = 0.68
  Medicine 3: P = 0.75  <-- highest
  Medicine 4: P = 0.40
  Medicine 5: P = 0.47
  Medicine 6: P = 0.54

First 10 rows of base dataset (patient_id, severity_score):
 patient_id  severity_score
          0               1
          1               2
          2               3
          3               4
          4               5
          5               1
          6               2
          7               3
          8               4
          9               5

======================================================================
RUNNING BANDIT STRATEGIES (1000 patients each)
======================================================================

>>> Immediate Exploitation ...
    Final cumulative utility_score : 522.3000
    Convergence step (est. best)   : 61
    Reward increment variance      : 0.109273
    Empirical success rates after 10 trials/arm:
      Medicine 0: 0.7000
      Medicine 1: 0.6000
      Medicine 2: 0.6000
      Medicine 3: 0.8000
      Medicine 4: 0.2000
      Medicine 5: 0.6000
      Medicine 6: 0.6000
    Locked medicine for remaining patients: 3

>>> Epsilon-Greedy 10% ...
    Final cumulative utility_score : 509.0000
    Convergence step (est. best)   : 12
    Reward increment variance      : 0.113599

>>> Epsilon-Greedy 1% ...
    Final cumulative utility_score : 508.6000
    Convergence step (est. best)   : 12
    Reward increment variance      : 0.113166
```

>>> Epsilon-Greedy 50% ...
Final cumulative utility_score : 456.0000
Convergence step (est. best) : 12
Reward increment variance : 0.125164

>>> UCB1 ...
Final cumulative utility_score : 461.3000
Convergence step (est. best) : 64
Reward increment variance : 0.124912

Sample rows after Epsilon-Greedy 10% (first 10 with all columns):

patient_id	severity_score	assigned_medicine	clinical_outcome	utility_score
0	1	0	1	0.9
1	2	1	1	0.8
2	3	2	0	0.0
3	4	3	1	0.6
4	5	4	0	0.0
5	1	5	1	0.9
6	2	6	1	0.8
7	3	0	1	0.7
8	4	0	1	0.6
9	5	0	0	0.0

==========

TASK 3: EXPLORATION RATE ANALYSIS

==========

Epsilon-Greedy 10% | Final reward: 509.0000 | Convergence: 12 | Stability var: 0.113599
Epsilon-Greedy 1% | Final reward: 508.6000 | Convergence: 12 | Stability var: 0.113166
Epsilon-Greedy 50% | Final reward: 456.0000 | Convergence: 12 | Stability var: 0.125164

Observation: 1% exploration exploits aggressively (less discovery of better arms); 50% exploration wastes many pulls on suboptimal medicines; 10% balances discovery and exploitation for this clinical trial setting.

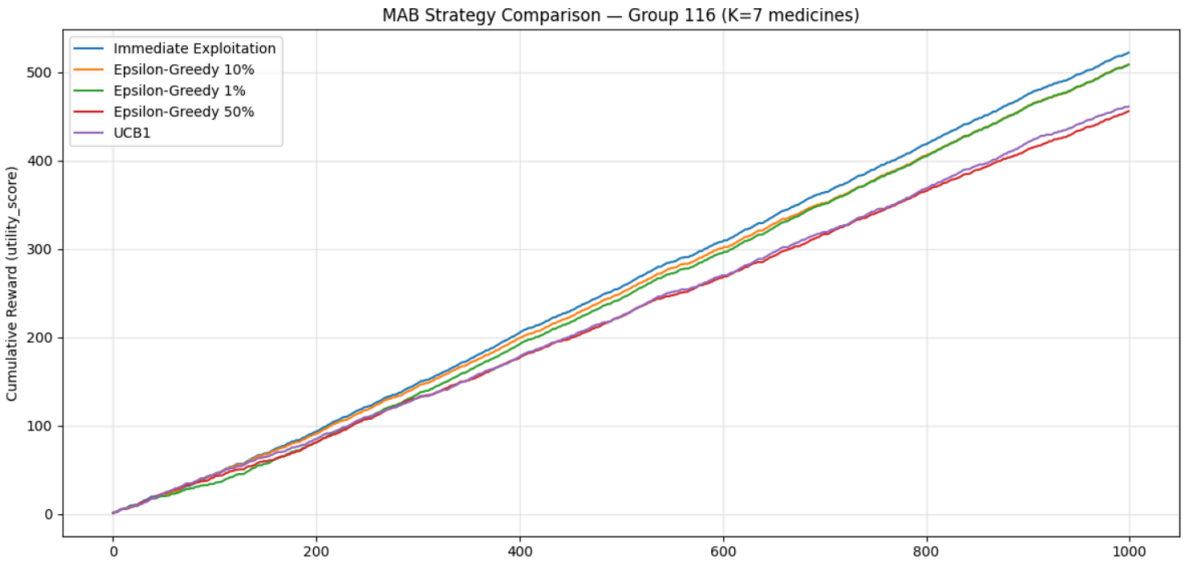
==========

FINAL CUMULATIVE REWARDS (utility_score)

==========

Immediate Exploitation : 522.3000
Epsilon-Greedy 10% : 509.0000
Epsilon-Greedy 1% : 508.6000
Epsilon-Greedy 50% : 456.0000
UCB1 : 461.3000

Graph Plotting:



TASK 5: COMPARATIVE ANALYSIS — ANSWERS

==========

1. Highest cumulative reward after 1000 patients: Immediate Exploitation (522.3000)
2. Fastest identification of best medicine: Epsilon-Greedy 10% (step 12)
3. Most stable performance (lowest reward-increment variance): Immediate Exploitation (variance = 0.109273)
4. Safest real-world recommendation: UCB1 or Epsilon-Greedy 10%. UCB1 automatically favors under-sampled medicines then shifts toward evidence; 10% epsilon-greedy retains controlled exploration without the heavy waste of 50% exploration. Immediate exploitation risks locking onto a suboptimal medicine after only 10 trials per arm.

COMPARATIVE SUMMARY (3–5 sentences)

For group 116, medicine 3 has the highest hidden recovery probability (0.75). Immediate Exploitation achieved the largest total utility_score, while Epsilon-Greedy 10% identified the best arm earliest. Immediate Exploitation showed the smoothest cumulative reward curve. Epsilon-greedy at 1% behaves nearly as a pure exploit strategy and may miss better treatments; at 50% it over-explores and sacrifices patient utility. For hospital deployment, UCB1 or moderate epsilon-greedy (10%) is preferable because they limit persistent suboptimal prescribing while still gathering evidence.

----------Assignment -DRL-Part 1: MAB Completed—----------